EXPERIMENT: LINUX LOGS AND USER ACTIVITY FORENSICS

1. Experiment Title

Linux Logs and User Activity Forensics

---

2. Objective

The objective of this experiment is to understand how Linux systems record operating-system and user activities and how investigators can extract, filter, correlate, and interpret those records during a forensic investigation.

Students will learn to:

1. Locate important Linux log files.
2. Understand the purpose and structure of **syslog**.
3. Investigate authentication activity using `auth.log`.
4. Investigate user activity using Bash history.
5. Query system events using `journalctl`.
6. Filter forensic evidence using `grep`.
7. Extract and summarize fields using `awk`.
8. Modify, normalize, or transform log text using `sed`.
9. Correlate multiple log sources to reconstruct user activity.
10. Identify successful and failed authentication attempts.
11. Identify suspicious commands and unusual activity.
12. Perform basic-to-advanced timeline analysis using Linux logs.

---

3. Learning Outcomes

After completing this experiment, students should be able to:

- Explain the difference between traditional syslog files and the systemd journal.
- Identify common Linux forensic log locations.
- Extract authentication events from system logs.
- Identify successful and failed login attempts.
- Determine which users accessed the system.
- Identify the source IP address associated with remote authentication attempts.
- Search Bash history for suspicious commands.
- Extract useful fields from semi-structured logs.

- Build a basic event timeline.
- Correlate evidence from multiple log sources.
- Recognize limitations of log-based investigation.

---

4. Tools Used

Tool	Primary Forensic Use
Syslog	Traditional Linux system and service logging
journalctl	Query and investigate systemd journal events
grep	Search and filter log entries
awk	Extract, process, and summarize fields
sed	Transform, clean, and manipulate log text
Bash History	Investigate commands executed by users

---

5. Operating Environment

Recommended environment:

- Ubuntu 22.04/24.04 LTS or Debian-based Linux
- Kali Linux can also be used
- Root/sudo privileges
- Terminal access
- Internet connection is optional

Important

Linux distributions do not always store logs in exactly the same locations.

For example, on Debian/Ubuntu systems, authentication events are commonly found in:

```
/var/log/auth.log
```

while some Red Hat-family systems commonly use:

```
/var/log/secure
```

The systemd journal may contain the same or additional information.

---

6. Forensic Safety Guidelines

During a real investigation, logs must be treated as evidence.

Do not modify the original evidence unnecessarily.

Use commands such as:

```
cat
less
grep
awk
sed
journalctl
```

for examination.

Whenever possible, create a working copy before performing extensive processing.

Example:

```
sudo cp /var/log/auth.log ~/auth.log.casecopy
```

Then work on:

```
~/auth.log.casecopy
```

Evidence Integrity

For a forensic investigation, record:

- Evidence source
- Acquisition date/time
- Investigator
- File path
- File size
- Hash value
- Analysis actions

Example:

```
sha256sum ~/auth.log.casecopy
```

Record the resulting SHA-256 value in the investigation notes.

---

7. Understanding Linux Logging

Linux systems generate logs for many types of activity.

Typical events include:

- User login
- User logout
- SSH connection
- Failed password attempts
- `sudo` execution
- Service startup
- Service shutdown
- Kernel events
- Network-related events
- Package installation
- System errors
- Authentication failures

A simplified investigation model is:

```
System/User Activity
        ↓
Logging Service
        ↓
Syslog / systemd journal
        ↓
Log Files / Journal Database
        ↓
Forensic Analysis
        ↓
Timeline / Findings
```

---

8. Common Linux Log Locations

First inspect the log directory:

```
ls -lah /var/log/
```

Look for:

```
syslog
auth.log
kern.log
messages
secure
journal/
```

Check whether the files exist:

```
ls -lh /var/log/syslog
ls -lh /var/log/auth.log
```

On systems that use `secure`:

```
ls -lh /var/log/secure
```

---

9. PART A — SYSLOG FORENSICS

9.1 What is Syslog?

Syslog is a standard logging mechanism used to record system and application events.

On many Debian/Ubuntu systems, a traditional system log is stored at:

```
/var/log/syslog
```

The format commonly contains:

```
Timestamp Hostname Process: Message
```

Example:

```
Sep 14 21:10:42 labhost systemd[1]: Started Session 12 of User student.
```

---

10. Step 1 — Check the Syslog File

Run:

```
ls -lh /var/log/syslog
```

If permitted:

```
sudo less /var/log/syslog
```

Navigation inside `less`:

```
Space      Next page
b          Previous page
/keyword   Search
n          Next match
q          Quit
```

---

11. Step 2 — Display Recent Syslog Entries

Display the last 20 entries:

```
sudo tail -n 20 /var/log/syslog
```

Display the last 50 entries:

```
sudo tail -n 50 /var/log/syslog
```

Follow the log in real time:

```
sudo tail -f /var/log/syslog
```

Press:

```
Ctrl+C
```

to stop.

---

12. Step 3 — Search Syslog With grep

Search for SSH:

```
sudo grep -i ssh /var/log/syslog
```

Search for systemd events:

```
sudo grep -i systemd /var/log/syslog
```

Search for errors:

```
sudo grep -i error /var/log/syslog
```

Search for warnings:

```
sudo grep -i warning /var/log/syslog
```

Search for multiple terms:

```
sudo grep -Ei 'error|warning|failed' /var/log/syslog
```

Forensic significance

This lets the investigator quickly isolate records related to:

- Errors

- Warnings
- Failed operations
- Specific services
- Specific keywords

---

13. Step 4 — Count Matching Events

Count occurrences of SSH-related events:

```
sudo grep -i ssh /var/log/syslog | wc -l
```

Count errors:

```
sudo grep -i error /var/log/syslog | wc -l
```

Count failed events:

```
sudo grep -i failed /var/log/syslog | wc -l
```

This provides a basic frequency analysis.

---

14. Step 5 — Search for a Specific Date

Example:

```
sudo grep 'Sep 14' /var/log/syslog
```

Search for a particular time:

```
sudo grep '21:10' /var/log/syslog
```

Search for a time range conceptually by matching prefixes:

```
sudo grep '21:' /var/log/syslog
```

---

15. Syslog Exercise

Task

Determine:

1. Whether SSH-related events exist.
2. How many error entries exist.
3. Whether any failed events are recorded.
4. Which services generated frequent messages.

Commands

```
sudo grep -i ssh /var/log/syslog
sudo grep -i error /var/log/syslog | wc -l
sudo grep -i failed /var/log/syslog | wc -l
```

Record your observations.

---

16. PART B — AUTHENTICATION LOG FORENSICS

16.1 What is auth.log?

Authentication logs contain records related to authentication and authorization activities.

Typical examples include:

- SSH login attempts
- Successful authentication
- Failed passwords
- `sudo` usage
- User session events
- Account-related activity

On Ubuntu/Debian:

```
/var/log/auth.log
```

Check:

```
sudo ls -lh /var/log/auth.log
```

---

17. Step 1 — Read auth.log

Use:

```
sudo less /var/log/auth.log
```

Display the newest entries:

```
sudo tail -n 30 /var/log/auth.log
```

---

18. Step 2 — Find SSH Activity

Run:

```
sudo grep -i ssh /var/log/auth.log
```

More specifically:

```
sudo grep 'sshd' /var/log/auth.log
```

This may show events such as:

```
sshd[1234]: Failed password for invalid user test from 192.168.1.50 port 42122
ssh2
```

or:

```
sshd[1234]: Accepted password for student from 192.168.1.20 port 42122 ssh2
```

---

19. Step 3 — Find Failed Login Attempts

Run:

```
sudo grep -i 'failed password' /var/log/auth.log
```

Find invalid users:

```
sudo grep -i 'invalid user' /var/log/auth.log
```

Find authentication failures:

```
sudo grep -Ei 'failed|failure|invalid user' /var/log/auth.log
```

---

20. Step 4 — Find Successful SSH Logins

Run:

```
sudo grep -i 'accepted' /var/log/auth.log
```

Typical records may include:

```
Accepted password
Accepted publickey
```

Search only accepted SSH connections:

```
sudo grep 'Accepted' /var/log/auth.log
```

---

21. Step 5 — Investigate sudo Activity

Search for `sudo`:

```
sudo grep -i 'sudo' /var/log/auth.log
```

A sudo record can reveal:

- User
- Time
- Terminal
- Command executed

Example structure:

```
student : TTY=pts/0 ; PWD=/home/student ; USER=root ; COMMAND=/usr/bin/id
```

This is valuable forensic evidence.

---

22. Step 6 — Identify Root Authentication

Search for root-related events:

```
sudo grep -i 'root' /var/log/auth.log
```

Be careful when interpreting this because the word `root` may occur for many reasons.

Prefer more specific searches:

```
sudo grep -i 'user root' /var/log/auth.log
```

---

23. Step 7 — Identify Remote Source IPs

Search failed SSH attempts:

```
sudo grep 'Failed password' /var/log/auth.log
```

Example:

```
Sep 14 21:22:15 server sshd[2101]: Failed password for root from 192.168.1.50
port 44210 ssh2
```

Important forensic fields:

```
Date
Time
Host
Process
Event
Username
Source IP
Source Port
Protocol
```

---

24. PART C — grep FOR FORENSIC ANALYSIS

24.1 Basic grep

Search for a word:

```
grep 'ssh' file.log
```

Case-insensitive:

```
grep -i 'ssh' file.log
```

Show line numbers:

```
grep -n 'ssh' file.log
```

Invert the match:

```
grep -v 'ssh' file.log
```

Count matches:

```
grep -c 'ssh' file.log
```

---

25. Recursive Log Searching

Search all files in `/var/log` for a term:

```
sudo grep -Rni 'failed password' /var/log
```

Explanation:

```
-R    Recursive
-n    Show line number
-i    Ignore case
```

---

26. Advanced grep Regular Expressions

Find failed or accepted authentication:

```
sudo grep -Ei 'failed password|accepted password|accepted publickey' /var/log/
auth.log
```

Search multiple suspicious indicators:

```
sudo grep -Ei 'failed|invalid|sudo|session opened|session closed' /var/log/
auth.log
```

Search for a private network source:

```
sudo grep -E '192\.168\.1\.[0-9]+' /var/log/auth.log
```

---

27. grep Forensic Exercise

Execute:

```
sudo grep -c 'Failed password' /var/log/auth.log
```

Then:

```
sudo grep -c 'Accepted' /var/log/auth.log
```

Compare:

```
Failed attempts
Successful attempts
```

Investigation Question

If there are many failures followed by one successful login from the same source, what could this indicate?

Possible interpretation:

- Password guessing
- Credential compromise
- Legitimate user who repeatedly mistyped a password
- Automated attack

Do not declare an attack based only on one log line. Correlate additional evidence.

---

28. PART D — AWK FOR FORENSIC EXTRACTION

28.1 Why use awk?

`awk` is useful for extracting fields from structured or semi-structured text.

Example:

```
awk '{print $1, $2, $3}' /var/log/auth.log
```

This extracts:

```
Month
Day
Time
```

from traditional syslog-style records.

---

29. Step 1 — Extract Timestamp

Run:

```
sudo awk '{print $1, $2, $3}' /var/log/auth.log
```

The output may resemble:

```
Sep 14 20:01:01
Sep 14 20:01:05
Sep 14 20:02:10
```

---

30. Step 2 — Extract Timestamp and Process

Run:

```
sudo awk '{print $1, $2, $3, $5}' /var/log/auth.log
```

The fifth field commonly contains a process identifier such as:

```
sshd[1234]:
```

---

31. Step 3 — Extract SSH-Related Records

Use:

```
sudo grep 'sshd' /var/log/auth.log | awk
'{print $1, $2, $3, $5, $6, $7, $8, $9, $10}'
```

This combines two forensic techniques:

```
grep → identify relevant records
awk  → extract important fields
```

---

32. Step 4 — Extract Failed SSH Events

Run:

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $1, $2, $3, $9, $11, $13}'
```

Depending on the exact log format, field positions can differ.

Therefore, always inspect representative records before relying on a field number.

---

33. Important awk Principle

Do not blindly assume that:

```
$9
$10
$11
```

always represent the same information.

First examine the raw record:

```
sudo grep 'Failed password' /var/log/auth.log | head
```

Then determine the field positions.

This is an important forensic practice because log formats can vary.

34. Step 5 — Count Authentication Events

Count failed SSH events:

```
sudo grep 'Failed password' /var/log/auth.log | awk '{count++} END {print count}'
```

Count accepted events:

```
sudo grep 'Accepted' /var/log/auth.log | awk '{count++} END {print count}'
```

---

35. Step 6 — Extract Source IP Addresses

One practical approach is to inspect the record structure and then select the appropriate field.

For example, for common OpenSSH messages:

```
sudo grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}'
```

Because `awk` supports:

```
NF = number of fields
```

using:

```
$(NF-3)
```

can sometimes be more robust than hard-coding a field number.

Always validate the result against actual log lines.

---

36. Step 7 — Count Repeated Source IP Addresses

Example:

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

This can produce output such as:

```
12 192.168.1.50
7  10.0.0.25
2  192.168.1.60
```

The result can help identify:

- Repeated authentication sources
- Potential password spraying
- Repeated attacks
- Internal administrative activity

Again, frequency alone does not prove malicious activity.

---

37. PART E — SED FOR FORENSIC TEXT PROCESSING

37.1 Why use sed?

`sed` is a stream editor.

It is useful for:

- Cleaning log output
- Replacing text
- Normalizing data
- Removing unwanted fields
- Preparing data for further analysis

It should normally be used on a **copy** when doing forensic processing.

---

38. Step 1 — Display Selected Lines

Example:

```
sudo sed -n '1,20p' /var/log/auth.log
```

Meaning:

```
-n       Suppress normal output
1,20p    Print lines 1 through 20
```

---

39. Step 2 — Display Only SSH Events

You can use:

```
sudo sed -n '/sshd/p' /var/log/auth.log
```

This prints lines matching:

```
sshd
```

---

40. Step 3 — Display Failed Authentication Events

```
sudo sed -n '/Failed password/p' /var/log/auth.log
```

---

41. Step 4 — Case-Insensitive Processing

Depending on the sed implementation, extended regex can be used:

```
sudo sed -n '/[Ff]ailed [Pp]assword/p' /var/log/auth.log
```

---

42. Step 5 — Replace Text in a Working Copy

Create a copy:

```
cp /var/log/auth.log ~/auth-analysis.log
```

Now replace a value:

```
sed 's/oldtext/newtext/g' ~/auth-analysis.log
```

The original file remains unchanged because the command only prints transformed output.

---

43. Step 6 — Remove Sensitive Information From an Analysis Copy

Suppose an IP address must be anonymized before sharing a training extract.

Example:

```
sed -E 's/([0-9]+\.[0-9]+\.[0-9]+)\.[0-9]+/\1.X/g' ~/auth-analysis.log
```

Example transformation:

```
192.168.1.50
```

becomes approximately:

```
192.168.1.X
```

This is useful when preparing classroom datasets.

---

44. sed + grep Combination

Example:

```
sudo grep 'Failed password' /var/log/auth.log |
sed 's/Failed password/FAIL/g'
```

This can normalize event names before further processing.

45. PART F — BASH HISTORY FORENSICS

45.1 What is Bash History?

Bash maintains a command history for interactive shell sessions.

Common history file:

```
~/.bash_history
```

Check:

```
ls -la ~/.bash_history
```

Read:

```
cat ~/.bash_history
```

or:

```
less ~/.bash_history
```

---

46. Important Bash History Limitation

Bash history is useful but should **not** be treated as a complete record of user activity.

A command may be absent because:

- History was disabled.
- History was cleared.
- The command was executed in another shell.
- The command was executed by another user.
- The shell did not write history before termination.
- The user intentionally altered history.
- A command was configured not to be stored.

Therefore:

```
Bash History ≠ Complete User Activity Record
```

Always correlate it with system logs and authentication logs.

---

47. Step 1 — Search History for Suspicious Commands

Search for network tools:

```
grep -Ei 'ssh|scp|nc|netcat|curl|wget' ~/.bash_history
```

Search for account manipulation:

```
grep -Ei 'useradd|usermod|passwd|chpasswd' ~/.bash_history
```

Search for privilege escalation:

```
grep -Ei 'sudo|su ' ~/.bash_history
```

Search for file deletion:

```
grep -Ei 'rm |rmdir|shred' ~/.bash_history
```

Search for archive creation:

```
grep -Ei 'tar |zip |gzip' ~/.bash_history
```

---

48. Step 2 — Look for Download Activity

```
grep -Ei 'wget|curl' ~/.bash_history
```

Possible forensic significance:

- Software download
- Script retrieval
- Remote file transfer
- Potential malware download

Do not assume malicious intent without additional evidence.

---

49. Step 3 — Look for Encoding or Obfuscation

Search for:

```
grep -Ei 'base64|openssl|xxd' ~/.bash_history
```

This may identify commands involving:

- Encoding
- Decoding
- Data transformation
- Obfuscated content

Again, the command alone is not sufficient to establish malicious intent.

---

50. Step 4 — Extract Numbered History

If the history file contains one command per line:

```
awk '{print NR, $0}' ~/.bash_history
```

This produces:

```
1 ls
2 cd /tmp
3 wget ...
4 chmod +x ...
```

---

51. Step 5 — Identify High-Risk Command Patterns

Use:

```
grep -Ei 'chmod|chown|passwd|useradd|usermod|sudo|su |rm |wget|curl|nc|python|
bash -c' ~/.bash_history
```

The objective is to identify commands requiring deeper examination.

---

52. PART G — JOURNALCTL FORENSICS

52.1 What is journalctl?

`journalctl` is the command-line interface for querying logs maintained by **systemd-journald**.

The journal can contain:

- System events
- Service events
- Authentication messages
- Kernel messages
- Boot records
- User sessions
- Application events

Check whether it is available:

```
journalctl --version
```

---

53. Step 1 — View the Journal

Run:

```
sudo journalctl
```

Because the output can be large, use:

```
sudo journalctl | less
```

---

54. Step 2 — View Recent Events

```
sudo journalctl -n 50
```

This displays the last 50 journal entries.

---

55. Step 3 — Follow the Journal Live

```
sudo journalctl -f
```

Press:

```
Ctrl+C
```

to stop.

This is similar to:

```
tail -f
```

for a traditional log file.

---

56. Step 4 — View Current Boot

```
sudo journalctl -b
```

This restricts results to the current boot.

---

57. Step 5 — View Previous Boot

List boots:

```
sudo journalctl --list-boots
```

Example concept:

```
-2 ...
-1 ...
 0 ...
```

Current boot is normally:

```
0
```

A previous boot can be queried using:

```
sudo journalctl -b -1
```

This is particularly useful when investigating events before a reboot.

---

58. Step 6 — Kernel Messages

Use:

```
sudo journalctl -k
```

This can help investigate:

- Hardware-related events
- Driver messages
- Kernel warnings
- Network interface events
- Device activity

---

59. Step 7 — Authentication-Related Journal Entries

Search for SSH:

```
sudo journalctl | grep -i ssh
```

Search specifically for sshd:

```
sudo journalctl -u ssh
```

On some systems the service may be:

```
sudo journalctl -u sshd
```

Check the available service name first when necessary.

---

60. Step 8 — Journal by Time

Example:

```
sudo journalctl --since "today"
```

Since yesterday:

```
sudo journalctl --since "yesterday"
```

Specific time:

```
sudo journalctl --since "2026-09-14 20:00:00"
```

Until a time:

```
sudo journalctl --until "2026-09-14 21:00:00"
```

Time range:

```
sudo journalctl --since "2026-09-14 20:00:00"
--until "2026-09-14 22:00:00"
```

---

61. Step 9 — Journal by Priority

Show errors:

```
sudo journalctl -p err
```

Warnings and above:

```
sudo journalctl -p warning
```

Common priority levels include:

```
0 emergency
1 alert
2 critical
3 error
4 warning
5 notice
6 informational
7 debug
```

---

62. Step 10 — Journal by User

If supported by the system metadata:

```
sudo journalctl _UID=1000
```

Replace `1000` with the UID under investigation.

Find a user's UID:

```
id username
```

Example:

```
id student
```

---

63. Step 11 — Journal by Process

Search for sshd messages:

```
sudo journalctl _COMM=sshd
```

Depending on the system, metadata may differ.

Inspect available fields with:

```
sudo journalctl -o verbose | less
```

This is an advanced forensic technique because it exposes journal metadata such as:

```
_SYSTEMD_UNIT
_UID
_GID
_PID
_COMM
_EXE
_HOSTNAME
_CMDLINE
```

---

64. PART H — COMBINING ALL TOOLS

A forensic investigation becomes more useful when multiple tools are combined.

Example pipeline:

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $1, $2, $3, $(NF-3)}'
```

Conceptually:

```
auth.log
   ↓
grep
   ↓
Relevant events
   ↓
awk
   ↓
Extracted fields
```

---

65. Advanced Authentication Investigation

Step 1 — Find Failed SSH Logins

```
sudo grep 'Failed password' /var/log/auth.log
```

Step 2 — Extract Source Information

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $(NF-3)}'
```

Step 3 — Count Repeated Sources

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

Step 4 — Investigate the Top Source

Suppose the top source is:

```
192.168.1.50
```

Search for it:

```
sudo grep '192.168.1.50' /var/log/auth.log
```

Search all logs:

```
sudo grep -Rni '192.168.1.50' /var/log
```

Then check the journal:

```
sudo journalctl | grep '192.168.1.50'
```

---

66. Timeline Reconstruction

A practical forensic objective is to answer:

```
WHO?
WHEN?
FROM WHERE?
WHAT DID THEY DO?
WHAT HAPPENED AFTERWARD?
```

Example investigation:

Question 1 — When was authentication attempted?

```
sudo grep 'Failed password' /var/log/auth.log
```

Question 2 — Was there eventually a successful login?

```
sudo grep 'Accepted' /var/log/auth.log
```

Question 3 — What did the user execute?

```
grep -Ei 'sudo|wget|curl|chmod|useradd|passwd' ~/.bash_history
```

Question 4 — Did system events occur afterward?

```
sudo journalctl --since "today"
```

Question 5 — Were there errors or warnings?

```
sudo journalctl -p warning
```

---

67. Correlation Example

Consider these hypothetical records:

```
20:10:01 Failed password for admin from 192.168.1.50
20:10:03 Failed password for admin from 192.168.1.50
20:10:07 Accepted password for admin from 192.168.1.50
20:11:01 sudo: admin COMMAND=/usr/bin/id
20:11:40 sudo: admin COMMAND=/usr/bin/whoami
```

Possible timeline:

```
20:10
     ↓
Repeated failed authentication
     ↓
Successful authentication
     ↓
20:11
     ↓
Privilege-related commands
```

This should trigger deeper investigation.

However, the investigator should not conclude compromise solely from this sequence.

Additional evidence should be examined.

---

68. Advanced grep + awk + sed Pipeline

Example:

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $1, $2, $3, $(NF-3)}' |
sed 's/^/[FAILED-SSH] /'
```

Potential output:

```
[FAILED-SSH] Sep 14 20:10:01 192.168.1.50
[FAILED-SSH] Sep 14 20:10:03 192.168.1.50
```

This provides a basic normalized event stream.

---

69. Exporting an Analysis Dataset

Create a working file:

```
sudo grep 'Failed password' /var/log/auth.log
> ~/failed-ssh-events.txt
```

Extract selected fields:

```
awk '{print $1, $2, $3, $(NF-3)}'
~/failed-ssh-events.txt
> ~/failed-ssh-summary.txt
```

Inspect:

```
cat ~/failed-ssh-summary.txt
```

Calculate the hash:

```
sha256sum ~/failed-ssh-summary.txt
```

This allows the analyst to document exactly which working output was generated.

70. Investigating sudo Usage

Search:

```
sudo grep -i 'sudo:' /var/log/auth.log
```

Extract:

```
sudo grep -i 'sudo:' /var/log/auth.log |
awk '{print $1, $2, $3, $5, $6, $7}'
```

For more detail:

```
sudo grep -i 'sudo:' /var/log/auth.log
```

Identify:

```
Timestamp
Username
TTY
Working directory
Target user
Command
```

71. Investigating Session Start and End

Search:

```
sudo grep -Ei 'session opened|session closed' /var/log/auth.log
```

This can help establish:

```
Session opened
       ↓
User activity
```

```
       ↓
Session closed
```

Combine with SSH records:

```
sudo grep -Ei 'accepted|session opened|session closed' /var/log/auth.log
```

---

72. Investigating Account Creation

Search:

```
sudo grep -Ei 'useradd|adduser|new user' /var/log/auth.log
```

Also inspect the journal:

```
sudo journalctl | grep -Ei 'useradd|adduser|new user'
```

Potential forensic question:

> Was a new account created shortly after an unusual login?

---

73. Investigating Privilege Escalation Indicators

Search history:

```
grep -Ei 'sudo|su |pkexec|chmod|chown' ~/.bash_history
```

Search authentication logs:

```
sudo grep -Ei 'sudo|su:|pkexec' /var/log/auth.log
```

Search the journal:

```
sudo journalctl | grep -Ei 'sudo|su:|pkexec'
```

Correlation across all three sources is stronger than relying on one source.

---

74. Investigating File and Script Execution

Search Bash history:

```
grep -Ei '\.sh|python|perl|ruby|bash -c|./' ~/.bash_history
```

Search logs for suspicious script names if known:

```
sudo grep -Rni 'scriptname.sh' /var/log
```

Search the journal:

```
sudo journalctl | grep -i 'scriptname'
```

---

75. Detecting Potential Log Rotation

Linux logs may be rotated.

Check:

```
ls -lh /var/log/auth.log*
```

Possible files:

```
auth.log
auth.log.1
auth.log.2.gz
```

Similarly:

```
ls -lh /var/log/syslog*
```

Compressed older logs may require different handling.

For example:

```
zgrep 'Failed password' /var/log/auth.log.2.gz
```

This is particularly important in historical investigations.

---

76. Basic-to-Advanced Investigation Workflow

Level 1 — Beginner

Task

Locate the logs:

```
ls -lh /var/log/
```

Check:

```
/var/log/syslog
/var/log/auth.log
```

Read the newest events:

```
sudo tail -n 20 /var/log/syslog
sudo tail -n 20 /var/log/auth.log
```

Level 2 — Intermediate

Search for:

```
sudo grep -i ssh /var/log/auth.log
sudo grep -i failed /var/log/auth.log
sudo grep -i accepted /var/log/auth.log
sudo grep -i sudo /var/log/auth.log
```

Answer:

1. How many failed login attempts occurred?
2. How many successful logins occurred?
3. Which users were involved?
4. Which source addresses were observed?

Level 3 — Advanced

Use:

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

Identify the most frequent source.

Then:

```
sudo grep '<IP>' /var/log/auth.log
```

Correlate with:

```
sudo journalctl | grep '<IP>'
```

Then inspect:

```
grep -Ei 'sudo|wget|curl|chmod|useradd|passwd|ssh' ~/.bash_history
```

---

77. Advanced Case Study

Scenario

A Linux server administrator reports suspicious activity around 20:00–21:00.

You are asked to determine:

> Was there suspicious authentication activity, and what happened after authentication?

Investigation Step 1 — Search Authentication Failures

```
sudo grep -Ei 'Failed password|invalid user' /var/log/auth.log
```

Record:

- Time
- Username
- Source IP

Investigation Step 2 — Search Successful Authentication

```
sudo grep -Ei 'Accepted password|Accepted publickey' /var/log/auth.log
```

Determine whether successful authentication occurred after the failures.

Investigation Step 3 — Investigate sudo

```
sudo grep -i 'sudo:' /var/log/auth.log
```

Record commands executed through sudo.

Investigation Step 4 — Examine User History

```
grep -Ei 'sudo|wget|curl|chmod|useradd|passwd|ssh|nc' ~/.bash_history
```

Record suspicious-looking commands.

Investigation Step 5 — Examine Journal Events

```
sudo journalctl --since "20:00:00" --until "21:00:00"
```

Search for relevant events:

```
sudo journalctl --since "20:00:00" --until "21:00:00" |
grep -Ei 'ssh|sudo|authentication|failed|error'
```

Investigation Step 6 — Build a Timeline

Create a table:

Time	Source	Event	User	IP	Interpretation
20:10	auth.log	Failed login	admin	192.168.1.50	Authentication failure
20:11	auth.log	Accepted login	admin	192.168.1.50	Successful authentication
20:12	auth.log	sudo	admin	—	Privileged command
20:14	Bash history	wget	admin	—	File download
20:15	journal	service event	—	—	Correlation required

Students must fill this table using actual evidence.

---

78. Recommended Investigation Questions

Students should answer:

Authentication

1. What was the earliest failed authentication event?
2. What was the latest failed authentication event?
3. How many failed attempts occurred?
4. How many successful attempts occurred?
5. Which username was targeted most frequently?
6. Which source IP generated the most failures?

User Activity

1. Which user executed privileged commands?
2. What commands were executed with `sudo`?
3. Are suspicious commands present in Bash history?
4. Were downloads performed?

System Activity

1. Were errors recorded?

2. Were warnings recorded?
3. Was the system rebooted?
4. Which services generated relevant events?

Correlation

1. Did a successful login follow repeated failed attempts?
2. Did privileged activity occur after successful authentication?
3. Did suspicious Bash commands occur after login?
4. Are the log and history timestamps consistent?

---

79. Important Forensic Limitations

Students must understand that logs are not perfect.

Potential issues include:

Log deletion

An attacker with sufficient privileges may alter or delete logs.

Log rotation

Old events may have moved to rotated files.

Timestamp issues

Time synchronization may be inaccurate.

Missing Bash history

A user's commands may not appear in history.

Multiple logging systems

The same event may appear differently in syslog and journald.

Privilege limitations

Some logs require root privileges to read.

Log tampering

Logs stored locally are not necessarily trustworthy by themselves.

Therefore:

```
Evidence Reliability increases when multiple independent sources agree.
```

---

80. Best-Practice Correlation Strategy

Use the following order:

```
1. Authentication Logs
        ↓
2. Successful Login
        ↓
3. Session Creation
        ↓
4. sudo / Privilege Activity
        ↓
5. Bash History
        ↓
6. System/Service Events
        ↓
7. Timeline Correlation
        ↓
8. Final Finding
```

---

81. Quick Reference — Core Commands

Syslog

```
sudo less /var/log/syslog
sudo tail -n 20 /var/log/syslog
sudo grep -i ssh /var/log/syslog
sudo grep -i error /var/log/syslog
```

Authentication

```
sudo less /var/log/auth.log
sudo grep 'Failed password' /var/log/auth.log
```

```
sudo grep 'Accepted' /var/log/auth.log
sudo grep 'sudo:' /var/log/auth.log
```

journalctl

```
sudo journalctl
sudo journalctl -n 50
sudo journalctl -f
sudo journalctl -b
sudo journalctl --list-boots
sudo journalctl -b -1
sudo journalctl -p err
sudo journalctl --since "today"
```

grep

```
grep -i keyword file
grep -n keyword file
grep -c keyword file
grep -Ei 'word1|word2' file
sudo grep -Rni keyword /var/log
```

awk

```
awk '{print $1,$2,$3}' file
awk '{print NR,$0}' file
awk '{count++} END {print count}' file
```

sed

```
sed -n '1,20p' file
sed -n '/pattern/p' file
sed 's/old/new/g' file
```

Bash History

```
cat ~/.bash_history
grep -Ei 'sudo|ssh|wget|curl' ~/.bash_history
```

---

82. Practical Lab Tasks

Task 1 — Syslog Examination

Identify:

- 10 recent syslog entries
- 5 error-related entries
- SSH-related entries, if present

Commands:

```
sudo tail -n 10 /var/log/syslog
sudo grep -i error /var/log/syslog | head -n 5
sudo grep -i ssh /var/log/syslog
```

Task 2 — Authentication Investigation

Find:

```
Failed logins
Successful logins
Invalid users
sudo activity
```

Commands:

```
sudo grep -Ei 'Failed password|invalid user|Accepted|sudo:' /var/log/auth.log
```

Task 3 — Source IP Frequency

```
sudo grep 'Failed password' /var/log/auth.log |
awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

Identify the top three sources.

Task 4 — Bash History Investigation

```
grep -Ei 'sudo|ssh|wget|curl|chmod|useradd|passwd|rm ' ~/.bash_history
```

Classify each command as:

```
Normal
Administrative
Potentially Suspicious
```

Do not label a command malicious solely from its name.

Task 5 — Journal Investigation

Run:

```
sudo journalctl --list-boots
```

Then:

```
sudo journalctl -b
```

Search:

```
sudo journalctl -b | grep -Ei 'ssh|sudo|failed|error'
```

---

83. Advanced Task — Automated Event Summary

Run:

```
echo "===== FAILED SSH ====="
sudo grep -c 'Failed password' /var/log/auth.log

echo "===== SUCCESSFUL SSH ====="
sudo grep -c 'Accepted' /var/log/auth.log
```

```
echo "===== SUDO EVENTS ====="
sudo grep -c 'sudo:' /var/log/auth.log

echo "===== INVALID USERS ====="
sudo grep -c 'invalid user' /var/log/auth.log
```

This creates a simple investigation summary.

---

84. Advanced Task — Build an Investigator Report

Students should prepare a report containing:

A. Evidence Sources

```
/var/log/syslog
/var/log/auth.log
systemd journal
~/.bash_history
```

B. Authentication Findings

```
Failed attempts:
Successful attempts:
Most targeted user:
Most frequent source IP:
```

C. User Activity

```
sudo commands:
Downloads:
Account modification:
Network utilities:
Suspicious commands:
```

D. Timeline

Record:

```
Time
Event
User
Source
Evidence
Interpretation
```

E. Conclusion

The conclusion must distinguish between:

```
Observed Fact
Inference
Unconfirmed Hypothesis
```

Example:

```
Observed Fact:
Five failed SSH attempts were recorded from the same source IP.

Inference:
The source may have been repeatedly attempting authentication.

Unconfirmed Hypothesis:
The activity may represent password guessing.
```

This distinction is essential in forensic reporting.

---

85. Viva Questions

Basic

1. What is syslog?
2. What is `journalctl`?
3. What is the purpose of `/var/log/auth.log`?
4. Where is Bash history usually stored?
5. What does `grep` do?
6. What does `awk` do?
7. What does `sed` do?

Intermediate

1. What is the difference between syslog and journald?
2. How can you identify failed SSH authentication?
3. How can you identify successful SSH authentication?
4. How can you identify sudo activity?
5. What does `grep -i` do?
6. What does `grep -c` do?
7. What does `$NF` mean in awk?
8. Why should field positions be validated before using awk?

Advanced

1. How can authentication logs be correlated with Bash history?
2. How would you identify the most frequent failed-login source?
3. How would you investigate activity from a previous boot?
4. What are the limitations of Bash history?
5. Why should investigators preserve original logs?
6. What is log rotation?
7. Why is local logging alone insufficient for high-confidence attribution?
8. How can an investigator distinguish observed evidence from inference?
9. Why is timeline correlation important?
10. How could an attacker attempt to interfere with local logs?

---

86. Expected Learning Result

After completing the experiment, students should be able to perform the following workflow independently:

```
Locate Evidence
      ↓
Preserve Evidence
      ↓
Read Logs
      ↓
Filter With grep
      ↓
Extract Fields With awk
      ↓
Normalize/Transform With sed
      ↓
Query Journal With journalctl
      ↓
Inspect Bash History
      ↓
```

```
Correlate Events
      ↓
Construct Timeline
      ↓
Interpret Findings
      ↓
Prepare Forensic Report
```

---

87. Final Practical Assignment

Scenario

You are the forensic analyst investigating a Linux server after an administrator reports suspicious user activity.

Determine:

1. Whether failed authentication attempts occurred.
2. Whether a successful login occurred after failures.
3. Which source IP was involved.
4. Which account was targeted.
5. Whether privileged commands were executed.
6. Whether Bash history contains suspicious commands.
7. Whether systemd journal records corroborating events.
8. Whether the activity can be placed into a coherent timeline.

Minimum commands to use

The investigation must demonstrate the use of all five required tools/techniques:

```
grep
awk
sed
journalctl
syslog/auth.log
```

Required Deliverables

Students must submit:

1. Terminal screenshots.
2. Extracted log evidence.
3. Authentication analysis.

4. Bash history analysis.
5. Event timeline.
6. Findings.
7. Limitations.
8. Final conclusion.

---

88. Suggested Lab Record Format

Experiment Number

Experiment: Linux Logs and User Activity Forensics

Date

Student Name

Roll Number

Objective

To extract, filter, analyze, and correlate Linux system logs and user activity using syslog, journalctl, grep, awk, sed, and Bash history.

System Information

```
uname -a
```

Record:

```
OS:
Kernel:
Hostname:
Username:
```

Evidence Sources

_____
_____
_____

Commands Executed

Record the commands used during the investigation.

Observations

_____
_____
_____

Timeline

Time	Evidence Source	Event	User	Source/IP	Observation

Findings

_____
_____
_____

Conclusion

_____
_____
_____

---

89. Conclusion

Linux logs provide valuable evidence for reconstructing system and user activity.

In this experiment, investigators learned to combine:

```
Syslog
   +
auth.log
   +
journalctl
   +
Bash History
   +
grep
   +
awk
   +
sed
```

The key forensic principle is **correlation**.

A single log entry rarely provides enough evidence to establish what happened. Stronger conclusions can be developed by correlating:

```
Authentication
      +
User Session
      +
Privilege Use
      +
Command History
      +
System Events
      +
Timeline
```

A competent Linux forensic investigator should therefore avoid relying on one log source and should document both the evidence and the limitations of that evidence.

End of Experiment